

Exercise 1 — Bubble Sort / Selection Sort (General)

Using a static `int [ ]` array, sort the array in ascending order using both the bubble sort and selection sort algorithms.

Exercise 2 — Selection Sort / Bubble Sort

Implement the Selection Sort or Bubble Sort algorithm to sort the list of integers [34, 12, 25, 9, 78, 31]. Print the sorted list.

Exercise 3 — Selection Sort / Bubble Sort with Step Tracking

Write a program to implement the Selection/Bubble Sort algorithm and sort the list of integers [45, 11, 32, 5, 27, 19] in ascending order, then print the sorted array.

Optional bonus: Count and print the number of comparisons and swaps performed.

Exercise 4 — Selection Sort / Bubble Sort

Write a program to implement the Selection/Bubble Sort algorithm and sort the list of integers [29, 10, 14, 37, 13]. Print the sorted list.

Exercise 5 — Selection Sort (with Intermediate Steps)

Implement the Selection Sort algorithm to sort an array of integers in ascending order.

Input: An array of integers.

Output: - The original array before sorting. - Intermediate steps showing the array after each pass of Selection Sort. - The final sorted array in ascending order.

Exercise 6 — Insertion Sort / Merge Sort

Implement Insertion Sort or Merge Sort to sort the list of integers [45, 11, 32, 5, 27, 19] in ascending order, and print the sorted array.

Optional bonus: Count and print the number of comparisons and swaps (if any).

Exercise 7 — Linear Search

Implement the Linear Search algorithm to search for a number in the list of integers [45, 23, 67, 12, 34, 89, 50]. Take a number as input from the keyboard and return its index if found, otherwise return -1.

Exercise 8 — Binary Search

Implement Binary Search to find a number in the sorted array [2, 5, 8, 12, 16, 23, 38, 56, 72, 91].

- Input a number from the keyboard.
- Return the index (0-based) if found, otherwise return -1.

Note: Do not use a built-in library search function.

Exercise 9 — Binary Search

Implement the Binary Search algorithm to search for a number in the sorted array [3, 6, 9, 12, 15, 18, 21, 24]. Take a number as input from the keyboard and return its index if found, otherwise return -1.

Exercise 10 — Longest Palindromic Substring

Write a program to find the longest palindromic substring in a given string.

Input: A string of characters.

Output: - The longest palindromic substring in the given string. - If no palindromic substring exists (of length greater than 1), print “No palindromic substring found.”

Example: Input: “abacdfgdcabba”

Exercise 11 — Sorting Student Exam Scores (Bubble Sort)

Write a program that allows: - Entering the exam scores of N students ($N \leq 100$). - Using Bubble Sort to sort the list of scores in descending order. - After sorting, displaying: - The sorted list of scores. - The student with the highest score and the student with the lowest score. - The number of swaps performed during the sort.

Sample Input:

```
Enter number of students: 5
Student 1 score: 7.5
Student 2 score: 8.0
Student 3 score: 6.5
Student 4 score: 9.0
Student 5 score: 7.0
```

Sample Output:

```
Sorted scores (descending):
9.0 8.0 7.5 7.0 6.5
Highest score: 9.0
Lowest score: 6.5
Total number of swaps: 6
```

Exercise 12 — Warehouse Product Search (Linear Search + Binary Search)

Build a simple warehouse-management program: - Create an array containing product codes (already sorted in ascending order). - Allow the user to: 1. Search for a product by code using Linear Search — print the position found and the execution time. 2. Search for a product by code using Binary Search — print the position found and the execution time. - Compare the running time of the two algorithms (use `System.currentTimeMillis()` or a similar timer).

Sample Input:

Product codes in warehouse (sorted): [101, 203, 305, 407, 512, 678, 789]

Enter the product code to search for: 512

Sample Output:

Linear Search:

Found at position 4

Time: 0 ms

Binary Search:

Found at position 4

Time: 0 ms

Exercise 13 — Ranking Students & Searching (Selection Sort + Binary Search)

Write a program that:

1. Reads a list of names and scores for N students.
2. Uses Selection Sort to sort the list by score in descending order (if scores are equal, sort by name in ascending order).
3. After sorting, allows searching for a student by name using Binary Search (since the array is sorted by score rather than by name, you will need to adapt the search logic accordingly).

Sample Input:

Number of students: 4

Student 1: Lan - 8.5

Student 2: Minh - 9.0

Student 3: Hoa - 8.5

Student 4: An - 7.0

Sample Output:

Ranking after sorting:

1. Minh - 9.0

2. Lan - 8.5

3. Hoa - 8.5

4. An - 7.0

Enter name to search for: Lan

-> Found: Lan - 8.5, rank 2